

Volume 54

Enterprise Scaling, Resilience & Performance Architecture

Status: Core Engineering Architecture

Vision

Project Sentinel must scale from:

1 home user

without changing the core architecture.

Scaling Philosophy

Scale horizontally. Keep the kernel stable.

The platform grows by adding:

Workers

not by rewriting the system.

Core Scaling Layers

User Interface

Each layer can scale independently.

Worker Scaling

Workers handle:

Web assessment

Large organizations can add more workers as needed.

Enterprise Fleet Scaling

Example:

50,000 laptops

Project Sentinel handles this through:

Distributed collectors

Performance Goals

The platform should prioritize:

Fast dashboard loading

Resilience Model

If one component fails, the system should degrade gracefully.

Example:

One worker fails

High Availability

Enterprise deployments may support:

Multiple API instances

Resource Guardian

Project Sentinel should avoid disrupting users.

Examples:

Pause heavy work on laptop battery

Evidence Scaling

Evidence can become very large.

The platform needs:

Evidence compression

NEW CORE ENGINE

Capacity Planning Engine

Estimates required workers, storage, CPU, memory, and network capacity based on:

Number of assets

NEW CORE ENGINE

Resilience Engine

Monitors:

Worker failures

and recommends operational actions.

NEW CORE ENGINE

Performance Intelligence Engine

Tracks long-term performance:

PS-ADR-0049

Horizontal Scaling by Design

Status: Accepted

Decision

Project Sentinel shall scale horizontally through workers, collectors, queues, replicated services, and independent processing layers rather than through a single monolithic runtime.

Rationale

Horizontal scaling improves resilience, performance, maintainability, and deployment flexibility across home, enterprise, and government environments.

Strategic Principle

Project Sentinel must become more powerful as it grows, not more complicated.

Next: **Volume 55 - Implementation Roadmap: Kernel → MVP → Production → Enterprise.**

====[USER]=====

Good and continue

====[ASSISTANT]=====

Excellent.

This is probably the most important volume we have written so far.

Everything before this volume has been architecture.

This volume answers the question:

How do we actually build Project Sentinel?

One of the biggest reasons large software projects fail is that they try to build everything at once.

We won't.

We'll build exactly like Linux, PostgreSQL, Docker and Kubernetes evolved:

- Stable core
- Small iterations
- Continuous improvements
- Never rewrite the kernel

That perfectly matches your original vision.

PROJECT SENTINEL BLUEPRINT

Volume 55